Floating Dictionary v1.5 — Complete Source Listing

> Text-source snapshot of the Android project. PNG launcher artwork and the Gradle wrapper JAR are binary assets and remain in the accompanying GitHub ZIP.

.gitignore

Plain Text

```
.gradle/
/build/
/app/build/
/local.properties
*.iml
.idea/
.DS_Store
```

app/build.gradle

Plain Text

```
plugins {
    id 'com.android.application'
    id 'org.jetbrains.kotlin.android'
}

android {
    namespace 'com.example.floatingdictionary'
    compileSdk 35

    defaultConfig {
        applicationId 'com.example.floatingdictionary'
        minSdk 29
        targetSdk 29
        // Keep versionCode increasing; this public release is version 1.5.
        versionCode 8
        versionName '1.5'
    }

    compileOptions {
        sourceCompatibility JavaVersion.VERSION_1_8
        targetCompatibility JavaVersion.VERSION_1_8
```

```
    }

    kotlinOptions {
        jvmTarget = '1.8'
    }
}
```

app/src/main/AndroidManifest.xml

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW" />
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <service
            android:name=".FloatingService"
            android:exported="false" />
    </application>
</manifest>
```

app/src/main/java/com/example/floatingdictionary/AppPreferences.kt

Kotlin

```kotlin
package com.example.floatingdictionary

import android.content.Context
import android.content.res.Configuration

object AppPreferences {
    const val THEME_LIGHT = "light"
    const val THEME_DARK = "dark"
    const val THEME_SYSTEM = "system"

    private const val PREFS_NAME = "floating_dictionary_preferences"
    private const val KEY_THEME = "theme"
    private const val KEY_LARGE_TEXT = "large_result_text"
    private const val KEY_SHOW_SYNONYMS = "show_synonyms"

    private fun preferences(context: Context ) =
        context.applicationContext.getSharedPreferences(PREFS_NAME, Context.MOD

    fun theme(context: Context): String =
        preferences(context).getString(KEY_THEME, THEME_LIGHT) ?: THEME_LIGHT

    fun setTheme(context: Context, value: String) {
        preferences(context).edit().putString(KEY_THEME, value).apply()
    }

    fun largeResultText(context: Context): Boolean =
        preferences(context).getBoolean(KEY_LARGE_TEXT, false)

    fun setLargeResultText(context: Context, enabled: Boolean) {
        preferences(context).edit().putBoolean(KEY_LARGE_TEXT, enabled).apply()
    }

    fun showSynonyms(context: Context): Boolean =
        preferences(context).getBoolean(KEY_SHOW_SYNONYMS, true)

    fun setShowSynonyms(context: Context, enabled: Boolean) {
        preferences(context).edit().putBoolean(KEY_SHOW_SYNONYMS, enabled).appl
    }

    fun resolveTheme(context: Context): Int {
        val useDark = when (theme(context)) {
            THEME_DARK -> true
            THEME_SYSTEM -> {
                val nightMode = context.resources.configuration.uiMode and Conf
                nightMode == Configuration.UI_MODE_NIGHT_YES
            }
            else -> false
```

```
        }
        return if (useDark) R.style.AppThemeDark else R.style.AppTheme
    }
}
```

app/src/main/java/com/example/floatingdictionary/FloatingService.kt

Kotlin

```
package com.example.floatingdictionary

import android.app.Notification
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.app.Service
import android.content.Context
import android.content.Intent
import android.graphics.PixelFormat
import android.os.Build
import android.os.Handler
import android.os.IBinder
import android.os.Looper
import android.provider.Settings
import android.text.Html
import android.view.ContextThemeWrapper
import android.view.Gravity
import android.view.LayoutInflater
import android.view.MotionEvent
import android.view.View
import android.view.ViewConfiguration
import android.view.WindowManager
import android.view.inputmethod.EditorInfo
import android.view.inputmethod.InputMethodManager
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import org.json.JSONArray
import org.json.JSONObject
import java.io.BufferedReader
import java.io.InputStreamReader
import java.net.HttpURLConnection
import java.net.URL
import java.net.URLEncoder
import java.util.Locale
```

```kotlin
import java.util.concurrent.ExecutorService
import java.util.concurrent.Executors
import java.util.concurrent.TimeoutException
import java.util.concurrent.TimeUnit
import kotlin.math.abs

class FloatingService : Service() {

    companion object {
        private const val CHANNEL_ID = "floating_dictionary_service"
        private const val NOTIFICATION_ID = 1001
        const val ACTION_REFRESH_THEME = "com.example.floatingdictionary.REFRES

        @Volatile
        var isRunning: Boolean = false
            private set
    }

    private data class HttpPayload(val status: Int, val body: String)

    private data class ParsedDefinition(
        val text: String,
        val wiktionarySynonyms: List<String>
    )

    private data class LookupResult(
        val word: String,
        val definition: String,
        val synonyms: List<String>,
        val synonymsAvailable: Boolean,
        val definitionUrl: String
    )

    private lateinit var windowManager: WindowManager
    private var overlayView: View? = null
    private var overlayParams: WindowManager.LayoutParams? = null
    private var panelOpen = false
    private var lastLookup: LookupResult? = null
    private val mainHandler = Handler(Looper.getMainLooper())
    private val lookupExecutor: ExecutorService = Executors.newSingleThreadExec
    private val networkExecutor: ExecutorService = Executors.newFixedThreadPool

    override fun onCreate() {
        super.onCreate()
        isRunning = true
        windowManager = getSystemService(Context.WINDOW_SERVICE) as WindowManag
    }
```

```
    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int
        isRunning = true
        createNotificationChannel()
        startForeground(NOTIFICATION_ID, buildNotification())

        if (!Settings.canDrawOverlays(this)) {
            stopSelf()
            return START_NOT_STICKY
        }

        if (intent?.action == ACTION_REFRESH_THEME && overlayView != null) {
            removeOverlay()
            showOverlay()
        } else if (overlayView == null) {
            showOverlay()
        }
        return START_STICKY
    }

    private fun showOverlay() {
        val themedContext = ContextThemeWrapper(this, AppPreferences.resolveThe
        val root = LayoutInflater.from(themedContext).inflate(R.layout.floating_
        val bubble = root.findViewById<TextView>(R.id.floatingBubble)
        val panel = root.findViewById<View>(R.id.searchPanel)
        val wordInput = root.findViewById<EditText>(R.id.wordInput)
        val searchButton = root.findViewById<Button>(R.id.searchButton)
        val saveButton = root.findViewById<Button>(R.id.saveWordButton)
        val closeButton = root.findViewById<Button>(R.id.closeButton)
        val stopBubbleButton = root.findViewById<Button>(R.id.stopBubbleButton)
        val definitionText = root.findViewById<TextView>(R.id.definitionText)

        definitionText.textSize = if (AppPreferences.largeResultText(this)) 16f
        saveButton.isEnabled = false

        val overlayType = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            WindowManager.LayoutParams.TYPE_APPLICATION_OVERLAY
        } else {
            @Suppress("DEPRECATION")
            WindowManager.LayoutParams.TYPE_PHONE
        }
        val params = WindowManager.LayoutParams(
            WindowManager.LayoutParams.WRAP_CONTENT,
            WindowManager.LayoutParams.WRAP_CONTENT,
            overlayType,
            WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE or
                WindowManager.LayoutParams.FLAG_NOT_TOUCH_MODAL,
            PixelFormat.TRANSLUCENT
        ).apply {
```

```
            gravity = Gravity.TOP or Gravity.START
            x = dp(16)
            y = dp(120)
        }

        overlayView = root
        overlayParams = params
        try {
            windowManager.addView(root, params)
        } catch (_: Exception) {
            overlayView = null
            overlayParams = null
            stopSelf()
            return
        }

        val touchSlop = ViewConfiguration.get(this).scaledTouchSlop
        bubble.setOnTouchListener(object : View.OnTouchListener {
            private var downRawX = 0f
            private var downRawY = 0f
            private var startX = 0
            private var startY = 0
            private var moved = false

            override fun onTouch(view: View, event: MotionEvent): Boolean {
                val currentParams = overlayParams ?: return false
                when (event.action) {
                    MotionEvent.ACTION_DOWN -> {
                        downRawX = event.rawX
                        downRawY = event.rawY
                        startX = currentParams.x
                        startY = currentParams.y
                        moved = false
                        return true
                    }
                    MotionEvent.ACTION_MOVE -> {
                        val dx = event.rawX - downRawX
                        val dy = event.rawY - downRawY
                        if (abs(dx) > touchSlop || abs(dy) > touchSlop) moved =
                        if (moved) {
                            currentParams.x = startX + dx.toInt()
                            currentParams.y = startY + dy.toInt()
                            try {
                                windowManager.updateViewLayout(root, currentPar
                            } catch (_: Exception) {
                                // The overlay may be removed while the user is
                            }
                        }
```

```
                        return true
                    }
                    MotionEvent.ACTION_UP -> {
                        if (!moved) togglePanel(panel, wordInput)
                        return true
                    }
                    MotionEvent.ACTION_CANCEL -> return true
                }
                return true
            }
        })

        fun performSearch() {
            val word = wordInput.text.toString().trim()
            if (word.isEmpty()) {
                definitionText.text = "Type an English word to look it up."
            } else {
                lookupWord(word, definitionText, saveButton)
            }
        }

        searchButton.setOnClickListener { performSearch() }
        wordInput.setOnEditorActionListener { _, actionId, _ ->
            if (actionId == EditorInfo.IME_ACTION_SEARCH) {
                performSearch()
                true
            } else {
                false
            }
        }
        saveButton.setOnClickListener {
            val lookup = lastLookup ?: return@setOnClickListener
            if (SavedWordsStore.contains(this, lookup.word)) {
                SavedWordsStore.remove(this, lookup.word)
                saveButton.text = "Save word"
            } else {
                SavedWordsStore.save(this, lookup.word, lookup.definition, look
                saveButton.text = "Saved ✓"
            }
        }
        closeButton.setOnClickListener { closePanel(panel, wordInput) }
        stopBubbleButton.setOnClickListener {
            closePanel(panel, wordInput)
            stopSelf()
        }
    }

    private fun togglePanel(panel: View, wordInput: EditText) {
```

```
        if (panelOpen) {
            closePanel(panel, wordInput)
            return
        }
        val root = overlayView ?: return
        val params = overlayParams ?: return
        panel.visibility = View.VISIBLE
        panelOpen = true
        params.flags = params.flags and WindowManager.LayoutParams.FLAG_NOT_FOC
        try {
            windowManager.updateViewLayout(root, params)
        } catch (_: Exception) {
            return
        }
        wordInput.requestFocus()
        wordInput.post {
            val inputMethodManager = getSystemService(Context.INPUT_METHOD_SERV
            inputMethodManager.showSoftInput(wordInput, InputMethodManager.SHOW
        }
    }

    private fun closePanel(panel: View, wordInput: EditText) {
        val root = overlayView ?: return
        val params = overlayParams ?: return
        val inputMethodManager = getSystemService(Context.INPUT_METHOD_SERVICE)
        inputMethodManager.hideSoftInputFromWindow(wordInput.windowToken, 0)
        panel.visibility = View.GONE
        wordInput.clearFocus()
        panelOpen = false
        params.flags = params.flags or WindowManager.LayoutParams.FLAG_NOT_FOCUS
        try {
            windowManager.updateViewLayout(root, params)
        } catch (_: Exception) {
            // The service may be shutting down.
        }
    }

    private fun lookupWord(word: String, output: TextView, saveButton: Button)
        output.text = "Looking up the definition and synonyms…"
        saveButton.isEnabled = false
        saveButton.text = "Save word"
        lastLookup = null
        val includeSynonyms = AppPreferences.showSynonyms(this)

        lookupExecutor.execute {
            val definitionFuture = networkExecutor.submit<HttpPayload> {
                request(
                    "https://en.wiktionary.org/api/rest_v1/page/definition/${en
```

```
                "application/json"
            )
        }
        val synonymFuture = if (includeSynonyms) {
            networkExecutor.submit<List<String>?> { fetchDatamuseSynonyms(w
        } else {
            null
        }

        var definitionFailureText = "Could not reach Wiktionary. Check your
        val definitionPayload = try {
            definitionFuture.get(17, TimeUnit.SECONDS)
        } catch (exception: Exception) {
            definitionFuture.cancel(true)
            val cause = exception.cause ?: exception
            definitionFailureText = if (cause is TimeoutException || cause
                "The definition request timed out. Check your connection an
            } else {
                "Could not reach Wiktionary. Check your connection and try
            }
            null
        }
        val parsed = if (definitionPayload?.status == HttpURLConnection.HTT
            try {
                parseDefinition(definitionPayload.body)
            } catch (_: Exception) {
                null
            }
        } else {
            null
        }
        val synonymsFromDatamuse: List<String>? = try {
            synonymFuture?.get(13, TimeUnit.SECONDS)
        } catch (_: Exception) {
            synonymFuture?.cancel(true)
            null
        }

        val failureText = when {
            definitionPayload == null -> definitionFailureText
            definitionPayload.status == HttpURLConnection.HTTP_NOT_FOUND ->
            definitionPayload.status !in 200..299 -> "Wiktionary returned H
            parsed == null -> "Wiktionary returned no readable English defi
            else -> null
        }

        val lookup = if (parsed != null && failureText == null) {
            val mergedSynonyms = LinkedHashSet<String>()
```

```
                parsed.wiktionarySynonyms.forEach { mergedSynonyms.add(it) }
                (synonymsFromDatamuse ?: emptyList()).forEach { candidate ->
                    if (!candidate.equals(word, ignoreCase = true)) mergedSynon
                }
                val pageTitle = encodePath(word.replace(' ', '_'))
                LookupResult(
                    word = word,
                    definition = parsed.text,
                    synonyms = mergedSynonyms.take(12),
                    synonymsAvailable = synonymsFromDatamuse != null || parsed.
                    definitionUrl = "https://en.wiktionary.org/wiki/$pageTitle"
                 )
            } else {
                null
            }
            val display = if (lookup != null) {
                formatResult(lookup, includeSynonyms)
            } else {
                val synonymHint = if (includeSynonyms && !synonymsFromDatamuse.
                    "\n\nRelated synonyms from Datamuse: ${synonymsFromDatamuse
                } else {
                    ""
                }
                (failureText ?: "Lookup failed. Please try again.") + synonymHi
            }

            mainHandler.post {
                if (overlayView != null && output.isAttachedToWindow) {
                    output.text = display
                    lastLookup = lookup
                    saveButton.isEnabled = lookup != null
                    saveButton.text = if (lookup != null && SavedWordsStore.con
                        "Saved ✓"
                    } else {
                        "Save word"
                    }
                }
            }
        }
    }

    private fun request(url: String, accept: String): HttpPayload {
        val connection = URL(url).openConnection() as HttpURLConnection
        try {
            connection.requestMethod = "GET"
            connection.connectTimeout = 8_000
            connection.readTimeout = 10_000
            connection.setRequestProperty("Accept", accept)
```

```
            connection.setRequestProperty(
                "User-Agent",
                "FloatingDictionary/1.5 (Android; dictionary definitions and sy
            )
            val status = connection.responseCode
            val stream = if (status in 200..299) connection.inputStream else co
            val body = if (stream != null) {
                BufferedReader(InputStreamReader(stream, Charsets.UTF_8)).use {
            } else {
                ""
            }
            return HttpPayload(status, body)
        } finally {
            connection.disconnect()
        }
    }

    private fun parseDefinition(response: String): ParsedDefinition? {
        val entries = JSONObject(response).optJSONArray("en") ?: JSONArray()
        val definitions = mutableListOf<String>()
        val synonyms = LinkedHashSet<String>()

        for (entryIndex in 0 until entries.length()) {
            val entry = entries.optJSONObject(entryIndex) ?: continue
            val partOfSpeech = entry.optString("partOfSpeech", "").trim()
            val definitionArray = entry.optJSONArray("definitions") ?: JSONArra
            for (definitionIndex in 0 until definitionArray.length()) {
                if (definitions.size >= 7) break
                val definitionObject = definitionArray.optJSONObject(definition
                val html = definitionObject.optString("definition", "")
                val clean = Html.fromHtml(html, Html.FROM_HTML_MODE_LEGACY)
                    .toString()
                    .replace(Regex("\\s+"), " ")
                    .trim()
                if (clean.isNotBlank()) {
                    val prefix = if (partOfSpeech.isNotBlank()) "$partOfSpeech:
                    definitions.add("• $prefix$clean")
                }
                addJsonStrings(synonyms, definitionObject.optJSONArray("synonym
            }
            addJsonStrings(synonyms, entry.optJSONArray("synonyms"))
            if (definitions.size >= 7) break
        }
        if (definitions.isEmpty()) return null
        return ParsedDefinition(definitions.joinToString("\n"), synonyms.toList
    }

    private fun fetchDatamuseSynonyms(word: String): List<String>? {
```

```
        val encoded = URLEncoder.encode(word, "UTF-8")
        val response = request(
            "https://api.datamuse.com/words?rel_syn=$encoded&max=12",
            "application/json"
         )
        if (response.status !in 200..299) return null
        val array = JSONArray(response.body)
        val synonyms = LinkedHashSet<String>()
        for (index in 0 until array.length()) {
            val item = array.optJSONObject(index) ?: continue
            val value = item.optString("word", "").trim()
            if (value.isNotEmpty() && !value.equals(word, ignoreCase = true)) s
        }
        return synonyms.toList()
    }

    private fun formatResult(lookup: LookupResult, includeSynonyms: Boolean): S
        append(lookup.word)
        append("\n\nDEFINITION\n")
        append(lookup.definition)
        if (includeSynonyms) {
            append("\n\nSYNONYMS (Datamuse / WordNet)\n")
            if (lookup.synonyms.isEmpty()) {
                append(
                    if (lookup.synonymsAvailable) {
                        "No exact synonyms were returned for this word."
                    } else {
                        "The synonym service is temporarily unavailable. Try ag
                    }
                )
            } else {
                append(lookup.synonyms.joinToString(", "))
            }
        }
        append("\n\nDefinition source: English Wiktionary\n")
        append(lookup.definitionUrl)
        if (includeSynonyms) {
            append("\n\nSynonym source: Datamuse API\nhttps://www.datamuse.com/
        }
    }.trim()

    private fun addJsonStrings(destination: MutableSet<String>, values: JSONArr
        if (values == null) return
        for (index in 0 until values.length()) {
            val value = values.optString(index).trim()
            if (value.isNotEmpty()) destination.add(value)
        }
    }
```

```
    private fun encodePath(value: String): String =
        URLEncoder.encode(value, "UTF-8").replace("+", "%20")

    private fun dp(value: Int): Int = (value * resources.displayMetrics.density

    private fun createNotificationChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                CHANNEL_ID,
                "Floating Dictionary",
                NotificationManager.IMPORTANCE_LOW
            ).apply {
                description = "Keeps the floating dictionary bubble available"
            }
            val manager = getSystemService(Context.NOTIFICATION_SERVICE) as Not
            manager.createNotificationChannel(channel)
        }
    }

    private fun buildNotification(): Notification {
        val activityIntent = Intent(this, MainActivity::class.java)
        val pendingIntent = PendingIntent.getActivity(
            this,
            0,
            activityIntent,
            PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
        )
        return Notification.Builder(this, CHANNEL_ID)
            .setContentTitle("Floating Dictionary is running")
            .setContentText("Tap the floating bubble to search for a word")
            .setSmallIcon(android.R.drawable.ic_menu_search)
            .setContentIntent(pendingIntent)
            .setOngoing(true)
            .build()
    }

    private fun removeOverlay() {
        val oldView = overlayView
        if (oldView != null) {
            try {
                windowManager.removeViewImmediate(oldView)
            } catch (_: Exception) {
                // The view may already have been detached.
            }
        }
        overlayView = null
        overlayParams = null
```

```
        panelOpen = false
        lastLookup = null
    }

    override fun onDestroy() {
        removeOverlay()
        lookupExecutor.shutdownNow()
        networkExecutor.shutdownNow()
        stopForeground(true)
        isRunning = false
        super.onDestroy()
    }

    override fun onBind(intent: Intent?): IBinder? = null
}
```

app/src/main/java/com/example/floatingdictionary/MainActivity.kt

Kotlin

```
package com.example.floatingdictionary

import android.app.Activity
import android.app.AlertDialog
import android.content.Intent
import android.net.Uri
import android.os.Build
import android.os.Bundle
import android.provider.Settings
import android.util.TypedValue
import android.view.Gravity
import android.view.View
import android.widget.Button
import android.widget.LinearLayout
import android.widget.RadioButton
import android.widget.RadioGroup
import android.widget.ScrollView
import android.widget.Switch
import android.widget.TextView

class MainActivity : Activity() {

    private var startAfterPermission = false
    private lateinit var statusText: TextView
    private lateinit var actionButton: Button
```

```
    private lateinit var savedWordsContainer: LinearLayout
    private lateinit var savedWordsEmpty: TextView
    private lateinit var savedWordsCount: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        setTheme(AppPreferences.resolveTheme(this))
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        statusText = findViewById(R.id.permissionStatus)
        actionButton = findViewById(R.id.actionButton)
        savedWordsContainer = findViewById(R.id.savedWordsContainer)
        savedWordsEmpty = findViewById(R.id.savedWordsEmpty)
        savedWordsCount = findViewById(R.id.savedWordsCount)

        actionButton.setOnClickListener {
            if (Settings.canDrawOverlays(this)) {
                startFloatingService()
            } else {
                startAfterPermission = true
                val permissionIntent = Intent(
                    Settings.ACTION_MANAGE_OVERLAY_PERMISSION,
                    Uri.parse("package:$packageName")
                )
                startActivity(permissionIntent)
            }
        }
        findViewById<Button>(R.id.settingsButton).setOnClickListener { showSett
        updatePermissionStatus()
        refreshSavedWords()
    }

    override fun onResume() {
        super.onResume()
        updatePermissionStatus()
        refreshSavedWords()
        if (startAfterPermission && Settings.canDrawOverlays(this)) {
            startAfterPermission = false
            startFloatingService()
        }
    }

    private fun updatePermissionStatus() {
        val hasPermission = Settings.canDrawOverlays(this)
        statusText.text = if (hasPermission) {
            "Overlay access is ready. Start the bubble when you want to look up
        } else {
            "Allow display over other apps so the dictionary can appear while y
```

```
        }
        actionButton.text = if (hasPermission) "Start floating bubble" else "Al
    }

    private fun startFloatingService() {
        val serviceIntent = Intent(this, FloatingService::class.java)
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            startForegroundService(serviceIntent)
        } else {
            startService(serviceIntent)
        }
        statusText.text = "Floating Dictionary is active. Tap its bubble to sea
    }

    private fun refreshSavedWords() {
        if (!::savedWordsContainer.isInitialized) return
        val savedWords = SavedWordsStore.all(this)
        savedWordsContainer.removeAllViews()
        savedWordsCount.text = savedWords.size.toString()
        savedWordsEmpty.visibility = if (savedWords.isEmpty()) View.VISIBLE els

        savedWords.forEach { savedWord ->
            val card = LinearLayout(this).apply {
                orientation = LinearLayout.HORIZONTAL
                gravity = Gravity.CENTER_VERTICAL
                setPadding(dp(14), dp(10), dp(8), dp(10))
                background = getDrawable(R.drawable.card_background)
                isFocusable = true
                isClickable = true
            }
            val textColumn = LinearLayout(this).apply {
                orientation = LinearLayout.VERTICAL
                setOnClickListener { showSavedWord(savedWord) }
            }
            val wordView = TextView(this).apply {
                text = savedWord.word
                textSize = 17f
                setTextColor(colorFromTheme(android.R.attr.textColorPrimary))
                setTypeface(typeface, android.graphics.Typeface.BOLD)
            }
            val snippet = savedWord.definition
                .replace(Regex("\\s+"), " ")
                .take(112)
                .let { if (savedWord.definition.length > 112) "$it…" else it }
            val definitionView = TextView(this).apply {
                text = snippet.ifBlank { "Tap to view saved lookup" }
                textSize = 13f
                setTextColor(colorFromTheme(android.R.attr.textColorSecondary))
```

```
                maxLines = 2
                setPadding(0, dp(3), 0, 0)
            }
            textColumn.addView(wordView)
            textColumn.addView(definitionView)
            textColumn.layoutParams = LinearLayout.LayoutParams(0, LinearLayout

            val removeButton = Button(this).apply {
                text = "Remove"
                isAllCaps = false
                minWidth = dp(72)
                setOnClickListener {
                    SavedWordsStore.remove(this@MainActivity, savedWord.word)
                    refreshSavedWords()
                }
            }
            card.addView(textColumn)
            card.addView(removeButton)
            card.setOnClickListener { showSavedWord(savedWord) }
            val params = LinearLayout.LayoutParams(
                LinearLayout.LayoutParams.MATCH_PARENT,
                LinearLayout.LayoutParams.WRAP_CONTENT
            ).apply { bottomMargin = dp(8) }
            savedWordsContainer.addView(card, params)
        }
    }

    private fun showSavedWord(savedWord: SavedWordsStore.SavedWord) {
        val message = buildString {
            append(savedWord.definition.ifBlank { "No definition was stored." }
            if (savedWord.synonyms.isNotEmpty()) {
                append("\n\nSynonyms\n")
                append(savedWord.synonyms.joinToString(", "))
            }
        }
        AlertDialog.Builder(this)
            .setTitle(savedWord.word)
            .setMessage(message)
            .setPositiveButton("Remove from saved") { _, _ ->
                SavedWordsStore.remove(this, savedWord.word)
                refreshSavedWords()
            }
            .setNegativeButton("Close", null)
            .show()
    }

    private fun showSettings() {
        val content = LinearLayout(this).apply {
```

```
            orientation = LinearLayout.VERTICAL
            setPadding(dp(22), dp(8), dp(22), dp(8))
        }
        val scroll = ScrollView(this).apply { addView(content) }

        content.addView(sectionLabel("Appearance"))
        val themeValues = listOf(
            AppPreferences.THEME_LIGHT to "Light",
            AppPreferences.THEME_DARK to "Dark",
            AppPreferences.THEME_SYSTEM to "Follow device"
        )
        val themeGroup = RadioGroup(this).apply { orientation = RadioGroup.VERT
        var selectedThemeId = View.NO_ID
        themeValues.forEach { (value, label) ->
            val radio = RadioButton(this).apply {
                id = View.generateViewId()
                text = label
                setTextColor(colorFromTheme(android.R.attr.textColorPrimary))
                if (value == AppPreferences.theme(this@MainActivity)) selectedT
            }
            themeGroup.addView(radio)
        }
        if (selectedThemeId != View.NO_ID) themeGroup.check(selectedThemeId)
        content.addView(themeGroup)

        content.addView(sectionLabel("Reading"))
        val largeTextSwitch = Switch(this).apply {
            text = "Use larger lookup text"
            setTextColor(colorFromTheme(android.R.attr.textColorPrimary))
            isChecked = AppPreferences.largeResultText(this@MainActivity)
        }
        content.addView(largeTextSwitch)

        val synonymsSwitch = Switch(this).apply {
            text = "Show synonyms in search results"
            setTextColor(colorFromTheme(android.R.attr.textColorPrimary))
            isChecked = AppPreferences.showSynonyms(this@MainActivity)
        }
        content.addView(synonymsSwitch)

        val clearButton = Button(this).apply {
            text = "Clear all saved words"
            isAllCaps = false
            setOnClickListener { confirmClearSavedWords() }
        }
        content.addView(clearButton)

        val dialog = AlertDialog.Builder(this)
```

```
            .setTitle("Settings")
            .setView(scroll)
            .setNegativeButton("Cancel", null)
            .setPositiveButton("Apply", null)
            .create()
        dialog.setOnShowListener {
            dialog.getButton(AlertDialog.BUTTON_POSITIVE).setOnClickListener {
                val chosenTheme = themeValues.firstOrNull { (_, label) ->
                    val checked = themeGroup.checkedRadioButtonId
                    themeGroup.findViewById<RadioButton>(checked)?.text == labe
                }?.first ?: AppPreferences.THEME_LIGHT
                AppPreferences.setTheme(this, chosenTheme)
                AppPreferences.setLargeResultText(this, largeTextSwitch.isCheck
                AppPreferences.setShowSynonyms(this, synonymsSwitch.isChecked)
                dialog.dismiss()
                recreate()
                refreshBubbleThemeIfRunning()
            }
        }
        dialog.show()
    }

    private fun confirmClearSavedWords() {
        AlertDialog.Builder(this)
            .setTitle("Clear saved words?")
            .setMessage("This removes all words saved on this device.")
            .setNegativeButton("Cancel", null)
            .setPositiveButton("Clear") { _, _ ->
                SavedWordsStore.clear(this)
                refreshSavedWords()
            }
            .show()
    }

    private fun refreshBubbleThemeIfRunning() {
        if (!FloatingService.isRunning) return
        val intent = Intent(this, FloatingService::class.java)
            .setAction(FloatingService.ACTION_REFRESH_THEME)
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            startForegroundService(intent)
        } else {
            startService(intent)
        }
    }

    private fun sectionLabel(text: String): TextView = TextView(this).apply {
        this.text = text
        textSize = 15f
```

```
        setTextColor(colorFromTheme(android.R.attr.textColorPrimary))
        setTypeface(typeface, android.graphics.Typeface.BOLD)
        setPadding(0, dp(16), 0, dp(4))
    }

    private fun colorFromTheme(attribute: Int): Int {
        val value = TypedValue()
        theme.resolveAttribute(attribute, value, true)
        return if (value.resourceId != 0) getColor(value.resourceId) else value
    }

    private fun dp(value: Int): Int = (value * resources.displayMetrics.density
}
```

app/src/main/java/com/example/floatingdictionary/SavedWordsStore.kt

Kotlin

```
package com.example.floatingdictionary

import android.content.Context
import org.json.JSONArray
import org.json.JSONObject

object SavedWordsStore {
    data class SavedWord(
        val word: String,
        val definition: String,
        val synonyms: List<String>,
        val savedAt: Long
    )

    private const val PREFS_NAME = "floating_dictionary_saved_words"
    private const val KEY_WORDS = "words_json"
    private val lock = Any()

    fun all(context: Context): List<SavedWord> = synchronized(lock) {
        readUnsafe(context).sortedByDescending { it.savedAt }
    }

    fun contains(context: Context, word: String): Boolean = synchronized(lock)
        readUnsafe(context).any { it.word.equals(word.trim(), ignoreCase = true
    }

    fun save(context: Context, word: String, definition: String, synonyms: List
```

```
        val cleanWord = word.trim()
        if (cleanWord.isEmpty()) return@synchronized
        val words = readUnsafe(context).toMutableList()
        words.removeAll { it.word.equals(cleanWord, ignoreCase = true) }
        words.add(
            0,
            SavedWord(
                word = cleanWord,
                definition = definition.trim(),
                synonyms = synonyms.map { it.trim() }.filter { it.isNotEmpty() }
                savedAt = System.currentTimeMillis()
            )
        )
        writeUnsafe(context, words)
    }

    fun remove(context: Context, word: String) = synchronized(lock) {
        val words = readUnsafe(context).filterNot { it.word.equals(word.trim(),
        writeUnsafe(context, words)
    }

    fun clear(context: Context) = synchronized(lock) {
        writeUnsafe(context, emptyList())
    }

    private fun readUnsafe(context: Context): List<SavedWord> {
        val preferences = context.applicationContext
            .getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)
        val raw = preferences.getString(KEY_WORDS, "[]") ?: "[]"
        return try {
            val array = JSONArray(raw)
            buildList {
                for (index in 0 until array.length()) {
                    val item = array.optJSONObject(index) ?: continue
                    val word = item.optString("word", "").trim()
                    if (word.isEmpty()) continue
                    val synonymsArray = item.optJSONArray("synonyms") ?: JSONAr
                    val synonyms = buildList {
                        for (synonymIndex in 0 until synonymsArray.length()) {
                            val synonym = synonymsArray.optString(synonymIndex)
                            if (synonym.isNotEmpty()) add(synonym)
                        }
                    }
                    add(
                        SavedWord(
                            word = word,
                            definition = item.optString("definition", ""),
                            synonyms = synonyms,
```

```
                            savedAt = item.optLong("savedAt", 0L)
                        )
                    )
                }
            }
        } catch (_: Exception) {
            emptyList()
        }
    }

    private fun writeUnsafe(context: Context, words: List<SavedWord>) {
        val array = JSONArray()
        words.forEach { savedWord ->
            val synonyms = JSONArray()
            savedWord.synonyms.forEach { synonyms.put(it) }
            array.put(
                JSONObject()
                    .put("word", savedWord.word)
                    .put("definition", savedWord.definition)
                    .put("synonyms", synonyms)
                    .put("savedAt", savedWord.savedAt)
            )
        }
        context.applicationContext
            .getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)
            .edit()
            .putString(KEY_WORDS, array.toString())
            .apply()
    }
}
```

app/src/main/res/drawable/bubble_background.xml

XML

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android" android:shape
    <solid android:color="#315FAD" />
    <stroke android:width="2dp" android:color="?android:attr/colorBackgroundFlo
</shape>
```

app/src/main/res/drawable/card_background.xml

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android" android:shape
    <solid android:color="?android:attr/colorBackgroundFloating" />
    <corners android:radius="20dp" />
    <stroke android:width="1dp" android:color="?android:attr/colorControlHighli
</shape>
```

app/src/main/res/drawable/count_badge_background.xml

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android" android:shape
    <solid android:color="?android:attr/colorBackgroundFloating" />
    <stroke android:width="1dp" android:color="?android:attr/colorControlHighli
</shape>
```

app/src/main/res/drawable/panel_background.xml

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android" android:shape
    <solid android:color="?android:attr/colorBackgroundFloating" />
    <corners android:radius="20dp" />
    <stroke android:width="1dp" android:color="?android:attr/colorControlHighli
</shape>
```

app/src/main/res/layout/activity_main.xml

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="?android:attr/colorBackground"
    android:fillViewport="true"
    android:overScrollMode="ifContentScrolls">

    <LinearLayout
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:paddingStart="20dp"
        android:paddingTop="24dp"
        android:paddingEnd="20dp"
        android:paddingBottom="28dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:gravity="center_vertical"
            android:orientation="horizontal">

            <ImageView
                android:layout_width="52dp"
                android:layout_height="52dp"
                android:contentDescription="@string/app_name"
                android:scaleType="centerInside"
                android:src="@mipmap/ic_launcher" />

            <LinearLayout
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_marginStart="12dp"
                android:layout_weight="1"
                android:orientation="vertical">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="@string/app_name"
                    android:textColor="?android:attr/textColorPrimary"
                    android:textSize="20sp"
                    android:textStyle="bold" />

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="WORD LOOKUP COMPANION"
                    android:textColor="?android:attr/textColorSecondary"
                    android:textSize="11sp"
                    android:letterSpacing="0.08" />
            </LinearLayout>

            <Button
                android:id="@+id/settingsButton"
                android:layout_width="wrap_content"
                android:layout_height="44dp"
```

```
            android:minWidth="0dp"
            android:text="Settings"
            android:textAllCaps="false" />
    </LinearLayout>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="22dp"
        android:background="@drawable/card_background"
        android:elevation="2dp"
        android:orientation="vertical"
        android:padding="20dp">

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="@string/home_title"
            android:textColor="?android:attr/textColorPrimary"
            android:textSize="23sp"
            android:textStyle="bold" />

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="6dp"
            android:text="@string/home_subtitle"
            android:textColor="?android:attr/textColorSecondary"
            android:textSize="15sp" />

        <TextView
            android:id="@+id/permissionStatus"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="18dp"
            android:text="Checking overlay permission…"
            android:textColor="?android:attr/textColorSecondary"
            android:textSize="14sp" />

        <Button
            android:id="@+id/actionButton"
            android:layout_width="match_parent"
            android:layout_height="54dp"
            android:layout_marginTop="12dp"
            android:text="Allow overlay &amp;amp; continue"
            android:textAllCaps="false"
            android:textSize="16sp" />
    </LinearLayout>
```

```
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="20dp"
            android:background="@drawable/card_background"
            android:orientation="vertical"
            android:padding="18dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:gravity="center_vertical"
                android:orientation="horizontal">

                <TextView
                    android:layout_width="0dp"
                    android:layout_height="wrap_content"
                    android:layout_weight="1"
                    android:text="@string/saved_words_title"
                    android:textColor="?android:attr/textColorPrimary"
                    android:textSize="19sp"
                    android:textStyle="bold" />

                <TextView
                    android:id="@+id/savedWordsCount"
                    android:layout_width="32dp"
                    android:layout_height="32dp"
                    android:background="@drawable/count_badge_background"
                    android:gravity="center"
                    android:text="0"
                    android:textColor="?android:attr/colorAccent"
                    android:textSize="14sp"
                    android:textStyle="bold" />
            </LinearLayout>

            <TextView
                android:id="@+id/savedWordsEmpty"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="12dp"
                android:text="@string/saved_words_empty"
                android:textColor="?android:attr/textColorSecondary"
                android:textSize="14sp" />

            <LinearLayout
                android:id="@+id/savedWordsContainer"
                android:layout_width="match_parent"
```

```
                android:layout_height="wrap_content"
                android:layout_marginTop="12dp"
                android:orientation="vertical" />
        </LinearLayout>

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="18dp"
            android:gravity="center"
            android:text="Definitions from English Wiktionary · Synonyms powere
            android:textColor="?android:attr/textColorSecondary"
            android:textSize="12sp" />
    </LinearLayout>
</ScrollView>
```

app/src/main/res/layout/floating_layout.xml

XML

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/floatingRoot"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <TextView
        android:id="@+id/floatingBubble"
        android:layout_width="56dp"
        android:layout_height="56dp"
        android:background="@drawable/bubble_background"
        android:contentDescription="@string/bubble_description"
        android:elevation="6dp"
        android:gravity="center"
        android:text="D"
        android:textColor="#FFFFFF"
        android:textSize="24sp"
        android:textStyle="bold" />

    <LinearLayout
        android:id="@+id/searchPanel"
        android:layout_width="300dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:background="@drawable/panel_background"
```

```
        android:elevation="10dp"
        android:orientation="vertical"
        android:padding="14dp"
        android:visibility="gone">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:gravity="center_vertical"
            android:orientation="horizontal">

            <TextView
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:text="Word lookup"
                android:textColor="?android:attr/textColorPrimary"
                android:textSize="18sp"
                android:textStyle="bold" />

            <Button
                android:id="@+id/stopBubbleButton"
                android:layout_width="72dp"
                android:layout_height="42dp"
                android:contentDescription="Turn off the floating bubble"
                android:minWidth="0dp"
                android:minHeight="0dp"
                android:text="Turn off"
                android:textAllCaps="false"
                android:textSize="12sp" />

            <Button
                android:id="@+id/closeButton"
                android:layout_width="42dp"
                android:layout_height="42dp"
                android:contentDescription="Close dictionary panel"
                android:minWidth="0dp"
                android:minHeight="0dp"
                android:text="×"
                android:textAllCaps="false"
                android:textSize="22sp" />
        </LinearLayout>

        <EditText
            android:id="@+id/wordInput"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
```

```
            android:hint="Type an English word"
            android:imeOptions="actionSearch"
            android:inputType="text"
            android:maxLines="1"
            android:singleLine="true"
            android:textColor="?android:attr/textColorPrimary"
            android:textColorHint="?android:attr/textColorHint" />

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="6dp"
            android:orientation="horizontal">

            <Button
                android:id="@+id/searchButton"
                android:layout_width="0dp"
                android:layout_height="48dp"
                android:layout_weight="1"
                android:text="Search"
                android:textAllCaps="false" />

            <Button
                android:id="@+id/saveWordButton"
                android:layout_width="0dp"
                android:layout_height="48dp"
                android:layout_marginStart="8dp"
                android:layout_weight="1"
                android:enabled="false"
                android:text="Save word"
                android:textAllCaps="false" />
        </LinearLayout>

        <ScrollView
            android:layout_width="match_parent"
            android:layout_height="240dp"
            android:layout_marginTop="8dp"
            android:fillViewport="true"
            android:overScrollMode="ifContentScrolls">

            <TextView
                android:id="@+id/definitionText"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:autoLink="web"
                android:lineSpacingExtra="2dp"
                android:linksClickable="true"
                android:text="@string/definition_placeholder"
```

```
                android:textColor="?android:attr/textColorPrimary"
                android:textSize="14sp" />
        </ScrollView>
    </LinearLayout>
</LinearLayout>
```

app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml

XML

```
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@color/launcher_background" />
    <foreground android:drawable="@mipmap/ic_launcher_foreground" />
</adaptive-icon>
```

app/src/main/res/mipmap-anydpi-v26/ic_launcher_round.xml

XML

```
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@color/launcher_background" />
    <foreground android:drawable="@mipmap/ic_launcher_foreground" />
</adaptive-icon>
```

app/src/main/res/values/colors.xml

XML

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="launcher_background">#F5FAFD</color>
</resources>
```

app/src/main/res/values/strings.xml

XML

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
```

```
    <string name="app_name">Floating Dictionary</string>
    <string name="home_title">Your reading, uninterrupted</string>
    <string name="home_subtitle">Look up English words without leaving your cur
    <string name="saved_words_title">Saved words</string>
    <string name="saved_words_empty">Your saved words will appear here. Search
    <string name="settings_title">Settings</string>
    <string name="bubble_description">Floating Dictionary. Tap to search or dra
    <string name="definition_placeholder">Your definition and synonyms will app
</resources>
```

app/src/main/res/values/styles.xml

XML

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="AppTheme" parent="@android:style/Theme.Material.Light.NoAction
        <item name="android:fontFamily">sans</item>
        <item name="android:colorAccent">#315FAD</item>
        <item name="android:colorBackground">#F5F7FA</item>
        <item name="android:windowBackground">#F5F7FA</item>
        <item name="android:statusBarColor">#F5F7FA</item>
        <item name="android:navigationBarColor">#F5F7FA</item>
        <item name="android:windowLightStatusBar">true</item>
        <item name="android:windowActionModeOverlay">true</item>
    </style>

    <style name="AppThemeDark" parent="@android:style/Theme.Material.NoActionBa
        <item name="android:fontFamily">sans</item>
        <item name="android:colorAccent">#9FC2FF</item>
        <item name="android:colorBackground">#111820</item>
        <item name="android:windowBackground">#111820</item>
        <item name="android:statusBarColor">#111820</item>
        <item name="android:navigationBarColor">#111820</item>
        <item name="android:windowLightStatusBar">false</item>
        <item name="android:windowActionModeOverlay">true</item>
    </style>
</resources>
```

build.gradle

Plain Text

```
plugins {
    id 'com.android.application' version '8.6.1' apply false
    id 'org.jetbrains.kotlin.android' version '2.0.21' apply false
}
```

gradle.properties

Plain Text

```
org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
org.gradle.parallel=true
```

gradle/wrapper/gradle-wrapper.properties

Plain Text

```
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
distributionUrl=https\://services.gradle.org/distributions/gradle-8.7-bin.zip
networkTimeout=10000
validateDistributionUrl=true
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
```

gradlew

Bash

```
#!/bin/sh

#
# Copyright © 2015-2021 the original authors.
#
# Licensed under the Apache License, Version 2.0 (the "License" );
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#      https://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
```

```
# See the License for the specific language governing permissions and
# limitations under the License.
#

##############################################################################
#
#   Gradle start up script for POSIX generated by Gradle.
#
#   Important for running:
#
#   (1 ) You need a POSIX-compliant shell to run this script. If your /bin/sh i
#       noncompliant, but you have some other compliant shell such as ksh or
#       bash, then to run this script, type that shell name before the whole
#       command line, like:
#
#           ksh Gradle
#
#       Busybox and similar reduced shells will NOT work, because this script
#       requires all of these POSIX shell features:
#         * functions;
#         * expansions «$var», «${var}», «${var:-default}», «${var+SET}»,
#           «${var#prefix}», «${var%suffix}», and «$( cmd )»;
#         * compound commands having a testable exit status, especially «case»;
#         * various built-in commands including «command», «set», and «ulimit».
#
#   Important for patching:
#
#   (2) This script targets any POSIX shell, so it avoids extensions provided
#       by Bash, Ksh, etc; in particular arrays are avoided.
#
#       The "traditional" practice of packing multiple parameters into a
#       space-separated string is a well documented source of bugs and security
#       problems, so this is (mostly) avoided, by progressively accumulating
#       options in "$@", and eventually passing that to Java.
#
#       Where the inherited environment variables (DEFAULT_JVM_OPTS, JAVA_OPTS,
#       and GRADLE_OPTS) rely on word-splitting, this is performed explicitly;
#       see the in-line comments for details.
#
#       There are tweaks for specific operating systems such as AIX, CygWin,
#       Darwin, MinGW, and NonStop.
#
#   (3) This script is generated from the Groovy template
#       https://github.com/gradle/gradle/blob/HEAD/subprojects/plugins/src/main
#       within the Gradle project.
#
#       You can find Gradle at https://github.com/gradle/gradle/.
#
```

```
##############################################################################

# Attempt to set APP_HOME

# Resolve links: $0 may be a link
app_path=$0

# Need this for daisy-chained symlinks.
while
    APP_HOME=${app_path%"${app_path##*/}"}  # leaves a trailing /; empty if no
    [ -h "$app_path" ]
do
    ls=$( ls -ld "$app_path"  )
    link=${ls#*' -> '}
    case $link in             #(
      /*)   app_path=$link ;; #(
      *)    app_path=$APP_HOME$link ;;
    esac
done

# This is normally unused
# shellcheck disable=SC2034
APP_BASE_NAME=${0##*/}
# Discard cd standard output in case $CDPATH is set (https://github.com/gradle/
APP_HOME=$( cd "${APP_HOME:-./}" > /dev/null && pwd -P ) || exit

# Use the maximum available, or set MAX_FD != -1 to use that value.
MAX_FD=maximum

warn () {
    echo "$*"
} >&2

die () {
    echo
    echo "$*"
    echo
    exit 1
} >&2

# OS specific support (must be 'true' or 'false').
cygwin=false
msys=false
darwin=false
nonstop=false
case "$( uname )" in                #(
  CYGWIN* )         cygwin=true  ;; #(
  Darwin* )         darwin=true  ;; #(
```

```
  MSYS* | MINGW* )  msys=true    ;; #(
  NONSTOP* )        nonstop=true ;;
esac

CLASSPATH=$APP_HOME/gradle/wrapper/gradle-wrapper.jar


# Determine the Java command to use to start the JVM.
if [ -n "$JAVA_HOME" ] ; then
    if [ -x "$JAVA_HOME/jre/sh/java" ] ; then
        # IBM's JDK on AIX uses strange locations for the executables
        JAVACMD=$JAVA_HOME/jre/sh/java
    else
        JAVACMD=$JAVA_HOME/bin/java
    fi
    if [ ! -x "$JAVACMD" ] ; then
        die "ERROR: JAVA_HOME is set to an invalid directory: $JAVA_HOME

Please set the JAVA_HOME variable in your environment to match the
location of your Java installation."
    fi
else
    JAVACMD=java
    if ! command -v java >/dev/null 2>&1
    then
        die "ERROR: JAVA_HOME is not set and no 'java' command could be found i

Please set the JAVA_HOME variable in your environment to match the
location of your Java installation."
    fi
fi

# Increase the maximum file descriptors if we can.
if ! "$cygwin" && ! "$darwin" && ! "$nonstop" ; then
    case $MAX_FD in #(
      max*)
        # In POSIX sh, ulimit -H is undefined. That's why the result is checked
        # shellcheck disable=SC2039,SC3045
        MAX_FD=$( ulimit -H -n ) ||
            warn "Could not query maximum file descriptor limit"
    esac
    case $MAX_FD in  #(
      '' | soft) :;; #(
      *)
        # In POSIX sh, ulimit -n is undefined. That's why the result is checked
        # shellcheck disable=SC2039,SC3045
        ulimit -n "$MAX_FD" ||
            warn "Could not set maximum file descriptor limit to $MAX_FD"
```

```
    esac
fi

# Collect all arguments for the java command, stacking in reverse order:
#   * args from the command line
#   * the main class name
#   * -classpath
#   * -D...appname settings
#   * --module-path (only if needed)
#   * DEFAULT_JVM_OPTS, JAVA_OPTS, and GRADLE_OPTS environment variables.

# For Cygwin or MSYS, switch paths to Windows format before running java
if "$cygwin" || "$msys" ; then
    APP_HOME=$( cygpath --path --mixed "$APP_HOME" )
    CLASSPATH=$( cygpath --path --mixed "$CLASSPATH" )

    JAVACMD=$( cygpath --unix "$JAVACMD" )

    # Now convert the arguments - kludge to limit ourselves to /bin/sh
    for arg do
        if
            case $arg in                                #(
              -*)   false ;;                            # don't mess with optio
              /?*)  t=${arg#/} t=/${t%%/*}              # looks like a POSIX fi
                    [ -e "$t" ] ;;                      #(
              *)    false ;;
            esac
        then
            arg=$( cygpath --path --ignore --mixed "$arg" )
        fi
        # Roll the args list around exactly as many times as the number of
        # args, so each arg winds up back in the position where it started, but
        # possibly modified.
        #
        # NB: a `for` loop captures its iteration list before it begins, so
        # changing the positional parameters here affects neither the number of
        # iterations, nor the values presented in `arg`.
        shift                   # remove old arg
        set -- "$@" "$arg"      # push replacement arg
    done
fi


# Add default JVM options here. You can also use JAVA_OPTS and GRADLE_OPTS to p
DEFAULT_JVM_OPTS='"-Xmx64m" "-Xms64m"'

# Collect all arguments for the java command:
#   * DEFAULT_JVM_OPTS, JAVA_OPTS, JAVA_OPTS, and optsEnvironmentVar are not al
```

```
#     and any embedded shellness will be escaped.
#   * For example: A user cannot expect ${Hostname} to be expanded, as it is an
#     treated as '${Hostname}' itself on the command line.

set -- \
        "-Dorg.gradle.appname=$APP_BASE_NAME" \
        -classpath "$CLASSPATH" \
        org.gradle.wrapper.GradleWrapperMain \
        "$@"

# Stop when "xargs" is not available.
if ! command -v xargs >/dev/null 2>&1
then
    die "xargs is not available"
fi

# Use "xargs" to parse quoted args.
#
# With -n1 it outputs one arg per line, with the quotes and backslashes removed
#
# In Bash we could simply go:
#
#   readarray ARGS < <( xargs -n1 <<<"$var" ) &&
#   set -- "${ARGS[@]}" "$@"
#
# but POSIX shell has neither arrays nor command substitution, so instead we
# post-process each arg (as a line of input to sed) to backslash-escape any
# character that might be a shell metacharacter, then use eval to reverse
# that process (while maintaining the separation between arguments), and wrap
# the whole thing up as a single "set" statement.
#
# This will of course break if any of these variables contains a newline or
# an unmatched quote.
#

eval "set -- $(
        printf '%s\n' "$DEFAULT_JVM_OPTS $JAVA_OPTS $GRADLE_OPTS" |
        xargs -n1 |
        sed ' s~[^-[:alnum:]+,./:=@_]~\\&~g; ' |
        tr '\n' ' '
    )" '"$@"'

exec "$JAVACMD" "$@"
```

gradlew.bat

Plain Text

```
@rem
@rem Copyright 2015 the original author or authors.
@rem
@rem Licensed under the Apache License, Version 2.0 (the "License");
@rem you may not use this file except in compliance with the License.
@rem You may obtain a copy of the License at
@rem
@rem      https://www.apache.org/licenses/LICENSE-2.0
@rem
@rem Unless required by applicable law or agreed to in writing, software
@rem distributed under the License is distributed on an "AS IS" BASIS,
@rem WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
@rem See the License for the specific language governing permissions and
@rem limitations under the License.
@rem

@if "%DEBUG%"=="" @echo off
@rem ##########################################################################
@rem
@rem  Gradle startup script for Windows
@rem
@rem ##########################################################################

@rem Set local scope for the variables with windows NT shell
if "%OS%"=="Windows_NT" setlocal

set DIRNAME=%~dp0
if "%DIRNAME%"=="" set DIRNAME=.
@rem This is normally unused
set APP_BASE_NAME=%~n0
set APP_HOME=%DIRNAME%

@rem Resolve any "." and ".." in APP_HOME to make it shorter.
for %%i in ("%APP_HOME%" ) do set APP_HOME=%%~fi

@rem Add default JVM options here. You can also use JAVA_OPTS and GRADLE_OPTS t
set DEFAULT_JVM_OPTS="-Xmx64m" "-Xms64m"

@rem Find java.exe
if defined JAVA_HOME goto findJavaFromJavaHome

set JAVA_EXE=java.exe
%JAVA_EXE% -version >NUL 2>&1
if %ERRORLEVEL% equ 0 goto execute

echo. 1>&2
echo ERROR: JAVA_HOME is not set and no 'java' command could be found in your P
```

```
echo. 1>&2
echo Please set the JAVA_HOME variable in your environment to match the 1>&2
echo location of your Java installation. 1>&2

goto fail

:findJavaFromJavaHome
set JAVA_HOME=%JAVA_HOME:"=%
set JAVA_EXE=%JAVA_HOME%/bin/java.exe

if exist "%JAVA_EXE%" goto execute

echo. 1>&2
echo ERROR: JAVA_HOME is set to an invalid directory: %JAVA_HOME% 1>&2
echo. 1>&2
echo Please set the JAVA_HOME variable in your environment to match the 1>&2
echo location of your Java installation. 1>&2

goto fail

:execute
@rem Setup the command line

set CLASSPATH=%APP_HOME%\gradle\wrapper\gradle-wrapper.jar


@rem Execute Gradle
"%JAVA_EXE%" %DEFAULT_JVM_OPTS% %JAVA_OPTS% %GRADLE_OPTS% "-Dorg.gradle.appname

:end
@rem End local scope for the variables with windows NT shell
if %ERRORLEVEL% equ 0 goto mainEnd

:fail
rem Set variable GRADLE_EXIT_CONSOLE if you need the _script_ return code inste
rem the _cmd.exe /c_ return code!
set EXIT_CODE=%ERRORLEVEL%
if %EXIT_CODE% equ 0 set EXIT_CODE=1
if not ""=="%GRADLE_EXIT_CONSOLE%" exit %EXIT_CODE%
exit /b %EXIT_CODE%

:mainEnd
if "%OS%"=="Windows_NT" endlocal

:omega
```

README.md

Markdown

```
# Floating Dictionary v1.5

A lightweight Android overlay for English word definitions while reading in oth

## What's new in v1.5

- Refreshed, light-first home screen with **Light**, **Dark**, and **Follow dev
- Readability setting for larger lookup text and an option to turn synonym look
- Saved words appear on the main page with their stored definitions and synonym
- Lookup panel displays Wiktionary definitions and requests synonyms separately
- The draggable bubble can still be turned off from inside the panel.

## Requirements and version

- Android 10 or newer (`minSdk 29`); project `targetSdk` is 29.
- Java 8 bytecode compatibility; build with Android Gradle Plugin 8.6.1, Kotlin
- Android app version: `1.5`; Android version code: `8`.

## Build

Open the project root in Android Studio and sync Gradle, or run this command wh

```sh
./gradlew assembleDebug
```

The result is `app/build/outputs/apk/debug/app-debug.apk`. This is a **debug-si

Use

1. Open **Floating Dictionary** and choose **Allow overlay & continue**.
2. Enable **Display over other apps** in Android settings. The floating bubble
3. Drag the bubble to move it, or tap it to open the lookup panel.
4. Enter an English word and press **Search**. Press **Save word** to keep the
5. Open **Settings** to select light, dark, or device theme; enlarge lookup tex
6. Press **Turn off** inside the panel to remove the bubble. Open the app and s

Word data sources

Definitions come from the experimental structured English Wiktionary endpoint:

When synonym lookup is enabled, the app additionally queries Datamuse with the
```

```
Datamuse asks publicly available apps to acknowledge its API in their documenta

## Project files

- `app/src/main/java/com/example/floatingdictionary/MainActivity.kt` — themed h
- `app/src/main/java/com/example/floatingdictionary/FloatingService.kt` — dragg
- `app/src/main/java/com/example/floatingdictionary/AppPreferences.kt` — theme,
- `app/src/main/java/com/example/floatingdictionary/SavedWordsStore.kt` — local
- `app/src/main/res/` — layouts, theme resources, and launcher icon assets.
- `branding/` — supplied logo and its launcher-size master.
- `Floating-Dictionary-source.md` — complete text source listing; image and Gra

## References

[1]: https://www.datamuse.com/api/ "Datamuse API documentation"
[2]: https://www.mediawiki.org/wiki/Wikimedia_REST_API "Wikimedia REST API docu
```

settings.gradle

Plain Text

```
pluginManagement {
    repositories {
        google( )
        mavenCentral()
        gradlePluginPortal()
    }
}

dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = 'Floating-Dictionary'
include ':app'
```